

Laboratorio 1 Base de datos avanzadas

Sistema de Gestión de Festivales de Música

Integrantes:
Andrés Candia Muñoz
Diego Cumare Calles
Joaquín Pino Barrios
Michel Vargas Carrasco
Gustavo Vásquez Ortiz

Introducción

Sistema de Gestión de Festivales de Música

Contexto: Una plataforma para gestionar festivales, recintos, escenarios, artistas, presentaciones (line-up), tipos de entrada, venta de tickets y control de acceso.

- Descripción avance de proyecto
- Diagrama de arquitectura
- Demostración de funcionalidades solicitadas
- Fragmentos de código
- Contribuciones (GitHub)
- Conclusiones

Descripción Avance de Proyecto

Planificación y reparto de responsabilidades:

- Definir el uso de ramas del repositorio
- Dividir las 10 exigencias del enunciado entre los participantes

Definición de base de datos:

- Analizar el contexto en conjunto
- Definir entidades y atributos

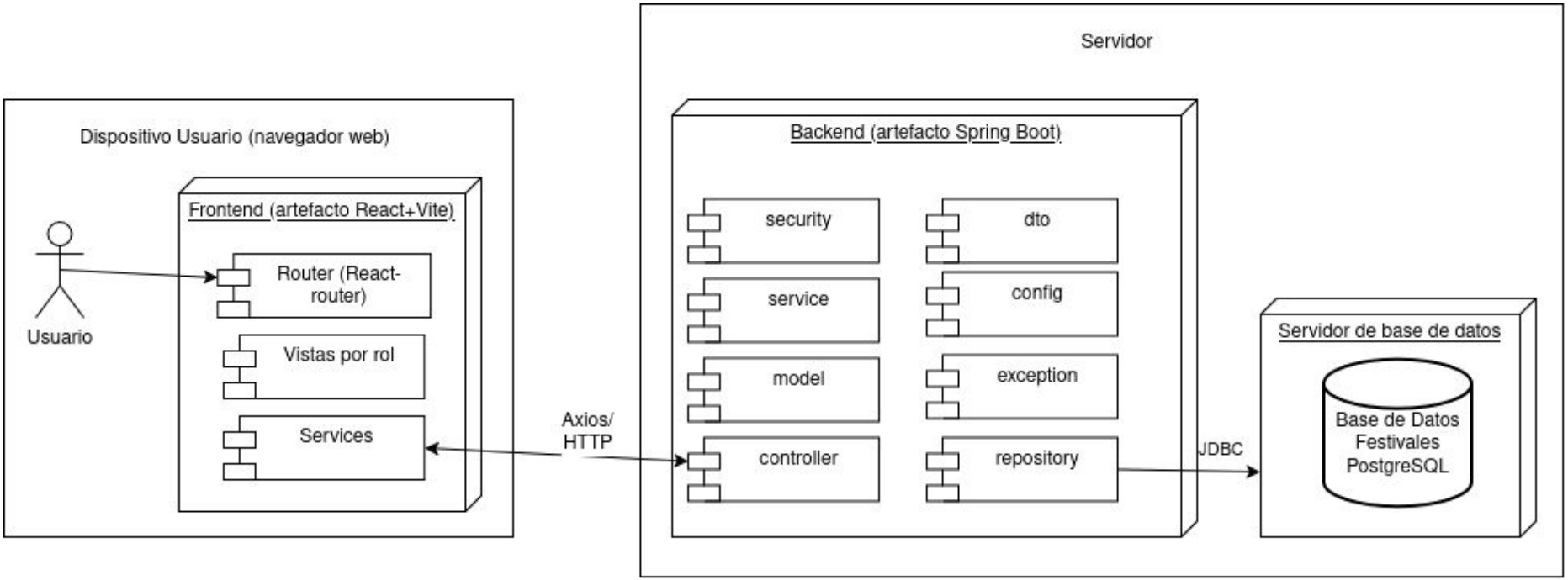
Implementación y pruebas:

- Desarrollar el código de manera individual
- Uso correcto de ramas y unir a rama “dev” solo lo que está probado y funcionando

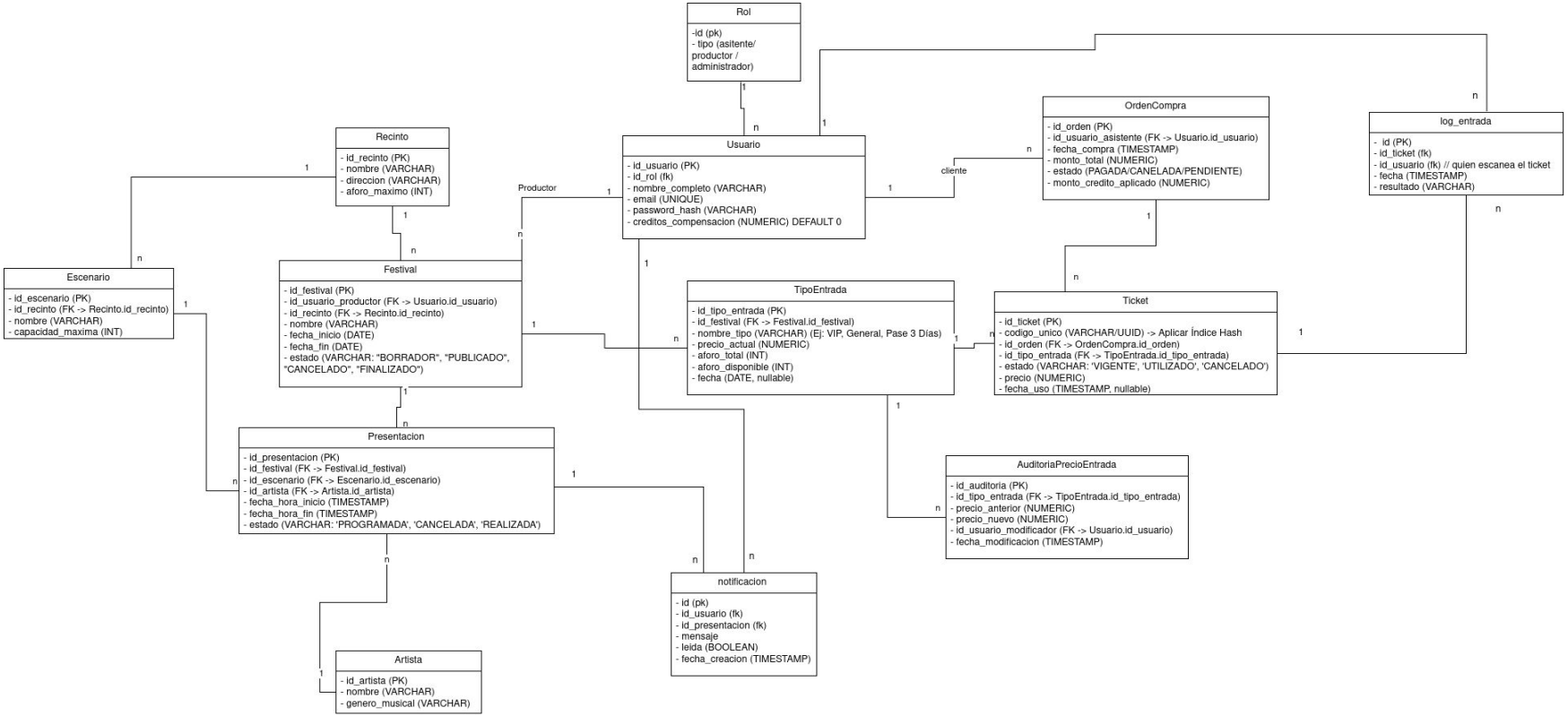
Verificación de exigencias:

- Se verifican las 10 exigencias del enunciado y se hace merge a la rama main

Diagrama de arquitectura



Modelo entidad Relación



Demostración de funciones

Demostración de funciones

1. API: CRUD completo de todas las entidades del modelo.
2. API/JWT: El Productor crea festivales y arma el line-up; el asistente compra entradas y solo puede ver sus propios tickets.
3. Procedimiento Almacenado 1: Comprar entradas: validar el cupo disponible del tipo de entrada, crear la orden, generar un ticket con código único por cada entrada y descontar el aforo en una sola transacción atómica.
4. Procedimiento Almacenado 2: Cancelar una presentación: marcarla como cancelada, generar notificaciones para todos los asistentes con entradas para ese día y asignarles un crédito de compensación.
5. Trigger 1: Impedir programar una presentación que se superponga en horario con otra en el mismo escenario, o que un mismo artista tenga dos presentaciones simultáneas.
6. Trigger 2: Registrar en una tabla de auditoría cada modificación del precio de un tipo de entrada, indicando precio anterior, precio nuevo, usuario y fecha.
7. Vista Materializada: "Ventas y Ocupación por Festival y Tipo de Entrada", con recaudación total y porcentaje de aforo vendido, refrescada diariamente al cierre de ventas.
8. Índices: Indexar el código único del ticket (Hash) y el ID del festival para acelerar la validación de accesos y los reportes.
9. API: Endpoint para obtener la grilla de presentaciones de un día del festival agrupada por escenario.
10. API: Endpoint de control de acceso que reciba el código de un ticket, verifique su validez y lo marque como utilizado, impidiendo su reutilización.

Fragmentos de código

```sql
CREATE TABLE rol (
    id SERIAL PRIMARY KEY,
    tipo VARCHAR(20) NOT NULL UNIQUE CHECK (tipo IN ('ASISTENTE', 'PRODUCTOR', 'ADMINISTRADOR'))
);

CREATE TABLE usuario (
    id_usuario BIGSERIAL PRIMARY KEY,
    id_rol INT NOT NULL REFERENCES rol(id),
    nombre_completo VARCHAR(150) NOT NULL,
    email VARCHAR(160) NOT NULL,
    password_hash TEXT NOT NULL,
    creditos_compensacion NUMERIC(12,2) NOT NULL DEFAULT 0 CHECK (creditos_compensacion >= 0)
);
CREATE UNIQUE INDEX uq_usuario_email ON usuario (lower(email));

CREATE TABLE recinto (
    id_recinto BIGSERIAL PRIMARY KEY,
    nombre VARCHAR(150) NOT NULL UNIQUE,
    direccion VARCHAR(250) NOT NULL,
    aforo_maximo INT NOT NULL CHECK (aforo_maximo > 0)
);

CREATE TABLE escenario (
    id_escenario BIGSERIAL PRIMARY KEY,
    id_recinto BIGINT NOT NULL REFERENCES recinto(id_recinto) ON DELETE CASCADE,
    nombre VARCHAR(120) NOT NULL,
    capacidad_maxima INT NOT NULL CHECK (capacidad_maxima > 0),
    CONSTRAINT uq_escenario_nombre UNIQUE (id_recinto, nombre)
);

CREATE TABLE artista (
    id_artista BIGSERIAL PRIMARY KEY,
    nombre VARCHAR(150) NOT NULL UNIQUE,
    genero_musical VARCHAR(80) NOT NULL
);

```

```
3    CREATE OR REPLACE FUNCTION fn_auditar_precio()
4    RETURNS TRIGGER AS $$
5    BEGIN
6        INSERT INTO auditoria_precio_entrada (id_tipo_entrada, precio_anterior, precio_nuevo, id_usuario_modificador)
7        VALUES (NEW.id_tipo_entrada, OLD.precio_actual, NEW.precio_actual, NULLIF(current_setting('app.user_id', true), '')::BIGINT);
8        RETURN NEW;
9    END;
10   $$ LANGUAGE plpgsql;
11
12   DROP TRIGGER IF EXISTS trg_auditoria_precio ON tipo_entrada;
13   CREATE TRIGGER trg_auditoria_precio
14   AFTER UPDATE OF precio_actual ON tipo_entrada
15   FOR EACH ROW
16   WHEN (OLD.precio_actual IS DISTINCT FROM NEW.precio_actual)
17   EXECUTE FUNCTION fn_auditar_precio();
```

```
CREATE MATERIALIZED VIEW mv_ventas AS
SELECT
    f.id_festival, -- ID del festival
    f.nombre AS festival_nombre, -- Nombre del festival
    te.nombre_tipo AS tipo_entrada, -- Nombre del tipo de entrada
    te.aforo_total, -- Aforo total
    (te.aforo_total - te.aforo_disponible) AS tickets_vendidos, -- Tickets vendidos
    ROUND(((te.aforo_total - te.aforo_disponible)::numeric / te.aforo_total) * 100, 2) AS porcentaje_ocupacion, -- Porcentaje de ocupación
    COALESCE(SUM(t.precio), 0) AS recaudacion_total -- COALESCE si el resultado de la suma es null devuelve un cero
FROM festival f
JOIN tipo_entrada te ON f.id_festival = te.id_festival
-- LEFT JOIN Mantiene los tipos de entrada aunque no hayan tickets vendidos en ese tipo
--  <> 'CANCELADO' importante para evitar ventas fantasma, entradas devueltas o echas credito
LEFT JOIN ticket t ON te.id_tipo_entrada = t.id_tipo_entrada AND t.estado <> 'CANCELADO'
GROUP BY f.id_festival, f.nombre, te.nombre_tipo, te.aforo_total, te.aforo_disponible;

-- 2. Función para refrescar los datos
CREATE OR REPLACE FUNCTION fn_refrescar_reporte_ventas()
RETURNS void AS $$
BEGIN
    REFRESH MATERIALIZED VIEW mv_ventas;
END;
$$ LANGUAGE plpgsql;
```

Repository

```java
@Repository
public class CancelacionRepository {

    /** Resultado escalar del procedimiento (sus parámetros INOUT). */
    public record Resultado(int notificados, BigDecimal creditoTotal) {
    }

    private final NamedParameterJdbcTemplate jdbc;

    public CancelacionRepository(NamedParameterJdbcTemplate jdbc) {
        this.jdbc = jdbc;
    }

    /**
     * Cancela la presentación, notifica y compensa. Debe ejecutarse dentro de una transacción.
     * Si no existe o no está PROGRAMADA, el procedimiento lanza RAISE EXCEPTION (409).
     */
    public Resultado cancelar(Long idPresentacion) {
        String sql = "CALL sp_cancelar_presentacion(:id, NULL::int, NULL::numeric)";
        return jdbc.queryForObject(sql, new MapSqlParameterSource("id", idPresentacion),
                (rs, i) -> new Resultado(rs.getInt("p_notificados"), rs.getBigDecimal("p_credito_total")));
    }

    /** Asistentes notificados por la cancelación con el crédito abonado a cada uno. */
    public List<CompensacionAsistenteResponse> findAsistentesNotificados(Long idPresentacion) {
        String sql = """
                SELECT u.id_usuario, u.nombre_completo, COALESCE(c.total, 0) AS credito
                FROM notificacion n
                JOIN usuario u ON u.id_usuario = n.id_usuario
                LEFT JOIN (SELECT id_usuario, SUM(monto) AS total
                           FROM compensacion WHERE id_presentacion = :id
                           GROUP BY id_usuario) c ON c.id_usuario = n.id_usuario
                WHERE n.id_presentacion = :id
                ORDER BY u.nombre_completo, u.id_usuario
                """;
        return jdbc.query(sql, new MapSqlParameterSource("id", idPresentacion),
                (rs, i) -> new CompensacionAsistenteResponse(rs.getLong("id_usuario"),
                        rs.getString("nombre_completo"), rs.getBigDecimal("credito")));
    }
}
```

Controller

```java
@Tag(name = "Escenarios")
@RestController
@RequestMapping("/api/escenarios")
public class EscenarioController {

    private final EscenarioService escenarioService;

    public EscenarioController(EscenarioService escenarioService) {
        this.escenarioService = escenarioService;
    }

    @Operation(summary = "Lista los escenarios (filtro opcional por id_recinto)")
    @Autenticado
    @GetMapping
    public List<EscenarioResponse> listar(@RequestParam(name = "id_recinto", required = false) Long idRecinto) {
        return escenarioService.listar(idRecinto);
    }

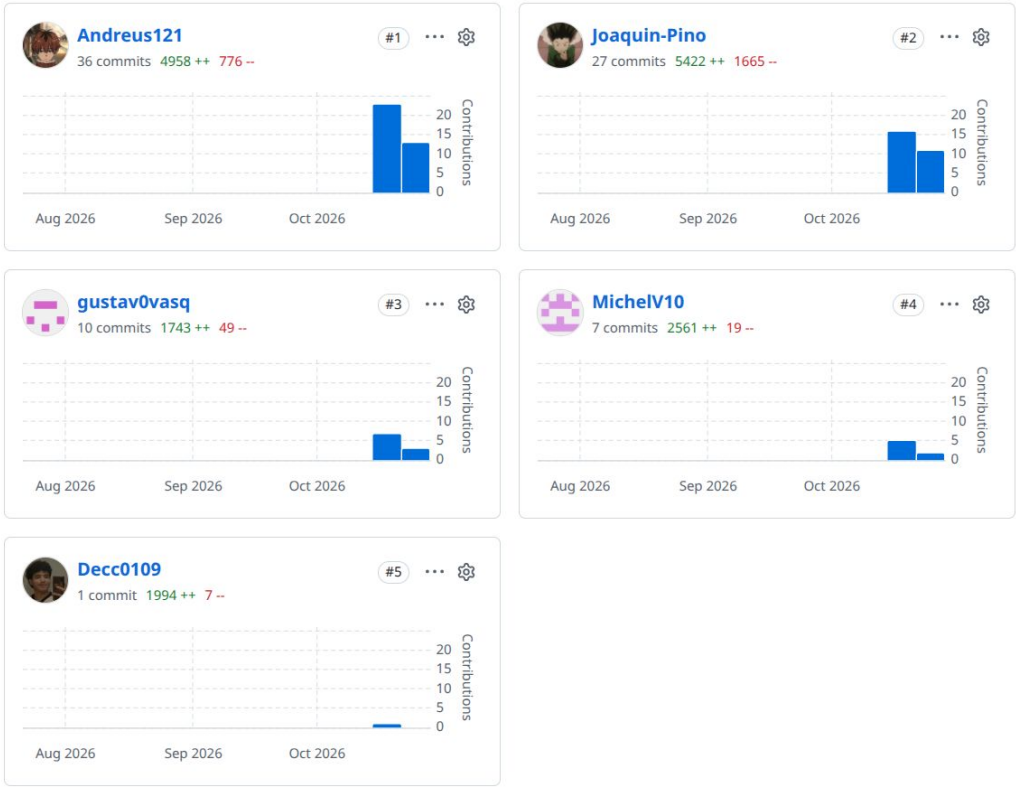
    @Operation(summary = "Obtiene un escenario por id")
    @Autenticado
    @GetMapping("/{id}")
    public EscenarioResponse obtener(@PathVariable Long id) {
        return escenarioService.obtener(id);
    }

    @Operation(summary = "Crea un escenario")
    @SoloProductorOAdmin
    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public EscenarioResponse crear(@Valid @RequestBody EscenarioRequest escenarioRequest) {
        return escenarioService.crear(escenarioRequest);
    }

    @Operation(summary = "Actualiza un escenario")
    @SoloProductorOAdmin
    @PutMapping("/{id}")
    public EscenarioResponse actualizar(@PathVariable Long id, @Valid @RequestBody EscenarioRequest escenarioRequest) {
        return escenarioService.actualizar(id, escenarioRequest);
    }

    @Operation(summary = "Elimina un escenario")
    @SoloProductorOAdmin
    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void eliminar(@PathVariable Long id) {
        escenarioService.eliminar(id);
    }
}
```

Contribuciones



Conclusiones

Puntos importantes del trabajo realizado:

- **Arquitectura desacoplada:** PostgreSQL + API REST sin ORM (JDBC) + frontend (React + Vite) que cubre el ciclo completo del festival.
- **Rendimiento:** la vista materializada de ventas y ocupación es la consulta más pesada, pero junto con índices para agilizar los reportes y la validación de accesos.
- **Seguridad:** JWT con roles (Productor, Asistente y Administrador), y el ticket queda invalidado tras su primer uso.
- **Consideraciones para próximo trabajo:** Definir un formato de commits y documentación de APIs con los JSON que usan estos mismos.

Muchas gracias por su atención.